

CVPR Project 5 — Final Report

KAT-Style In-Context Imitation Learning for Bimanual Manipulation

Sushant Subedi (ss17395)

1. Abstract

A KAT-style pipeline was implemented that maps top-view RGB-D observations and 14-D bimanual joint trajectories into discrete text tokens, prompts a local large language model (Ollama) for in-context imitation, parses the JSON response, and replays it in MuJoCo. Observation tokens use **DINO ViT-S/16 anchored keypoints** (best-buddy correspondences + FPS) with depth; action tokens use **uniform per-dimension quantization** of M waypoints.

Few-shot ICL is evaluated with explicit **seen / unseen** splits and four ablations (number of demos, K keypoints, M waypoints, and LLM model). The pipeline reaches the LLM correctly and produces well-formed JSON (100% parse-ok at M=20), but **task success on the held-out unseen split is 0/5** across every configuration. The decode-free GT replay confirms the simulator and replay paths are correct (5/5). The remaining gap is LLM reasoning capacity, consistent with the original KAT paper's reliance on GPT-4-class frontier models.

2. Method

2.1 Observation Tokens — Vision Only

Anchor selection extracts DINO ViT-S/16 patch descriptors for two frames of episode_0 (t=0 and t=T/2). Best-buddy correspondences are computed, then furthest-point sampling retains K spatially diverse anchors whose descriptors are cached.

Per-frame localization: each cached anchor descriptor is matched to the new image's patch grid by cosine NN → a stable index per anchor, shared across demos and queries. x,y are normalized image coordinates quantized to obs_bins (default 16); depth is clipped to [0, 2] m and quantized to obs_bins. No simulator ground truth is used.

```
OBS_START
KP 0 x=2 y=2 d=15
KP 1 x=14 y=8 d=12
...
OBS_END
```

2.2 Action Tokens

M waypoints are uniformly subsampled over an episode. A per-dimension uniform quantizer is fit over the demo folder (default bins=32). The 14 dimensions comprise: 6 left-arm joints, 1 left gripper, 6 right-arm joints, 1 right gripper. LG/RG ∈ {0,1} come from thresholding normalized gripper positions at 0.5.

```
TRAJ_START
WP[i=0] L=[47, 25, 45, 12, 27, 0] LG=0 R=[19, 21, 61, 40, 0, 22] RG=0
```

```
...  
TRAJ_END
```

2.3 ICL → JSON → Trajectory → Sim

The prompt is built from N demonstration blocks (OBS + TRAJ) followed by the query OBS. Ollama is called with temperature=0, num_predict=2048, and a structured JSON schema constraining the output to exactly M waypoints. The response is parsed, decoded via the quantizer, linearly upsampled to 400 sim steps, and replayed.

3. Dataset and Splits

25 episodes were collected with --seed 0 under the sim_transfer_cube_scripted task (downsample_rate=10, episode_len=400). Hard-linked subsets fix the seen/unseen split deterministically.

Split	Episodes	# Queries	Description
demos_5	0–4	5	In-context demos (main config)
demos_10	0–9	10	#Demos ablation
demos_20	0–19	20	#Demos ablation
test_5	20–24	5	Held-out — never in any demos_* folder

Seen split: leave-one-out on the demo folder; query is one demo, prompt holds the other N-1. **Unseen split:** held-out test folder; box poses from the same seeded run, but the prompt never includes the query.

4. Evaluation, Ablations, and Results

4.1 Decode-Free Upper Bound (Sanity Check)

GT replay reads raw /action keyframes from HDF5, linearly upsamples to 400 steps, and replays in sim with the correct env_state0. This is the upper bound for any perfect pipeline.

Folder	N	Success	Avg max_reward
demos_5	5	5 / 5 (100%)	4.00 / 4
test_5	5	5 / 5 (100%)	4.00 / 4

4.2 Main Configuration — Seen / Unseen

Model: gemma4:26b · K=10 · M=20 · bins=32 · episode_len=400

Split	N	parse_ok	Success	max_reward per query	Avg max_reward
seen (LOO, 0–4)	5	100%	0 / 5	0, 0, 1, 2, 1	0.80 / 4

Split	N	parse_ok	Success	max_reward per query	Avg max_reward
unseen (test, 20–24)	5	100%	0 / 5	0, 0, 0, 0, 0	0.00 / 4

Every response parses as valid JSON with exactly 20 waypoint objects. The seen split registers partial reward (avg 0.80) due to demo-conditional pattern matching; the unseen split is uniformly zero, confirming no generalizable mapping is learned.

4.3 Ablations (Unseen Split, queries 20–24)

(a) Number of In-Context Demos

#Demos	Demo IDs	parse_ok	Success	Avg max_reward
5	0–4	100%	0 / 5	0.00
10	0–9	100%	0 / 5	0.00
20	0–19	100%	0 / 5	0.00

More demos do not help. Prompt grows linearly but unseen rate stays at 0.

(b) Number of Keypoints K (M=20 fixed)

K	parse_ok	Success	Avg max_reward
5	100%	0 / 5	0.00
10	100%	0 / 5	0.00
20	100%	0 / 5	0.00

Doubling or halving K leaves unseen success at zero.

(c) Number of Action Waypoints M (K=10 fixed)

M	parse_ok	Success	Avg max_reward	Notes
10	100%	0 / 5	0.00	Shortest output
20	100%	0 / 5	0.00	Baseline
40	0%	0 / 5	0.00	All responses truncated — JSONDecodeError

M=40 is the first ablation where the system fails before the simulator runs: 40 waypoints x 14 ints exceeds Ollama's 2048-token budget, so responses are truncated and parsing fails.

(d) LLM Model Scale (K=10, M=20 fixed)

Model	~Params	parse_ok	Success	Avg max_reward
gemma4:26b	~26B	100%	0 / 5	0.00
llama3.2:latest	~3B	100%	0 / 5	0.00

A ~10x scale gap makes no difference. Both models comply with the JSON schema; neither solves the symbolic regression. This is consistent with the KAT paper's reliance on GPT-4-class frontier models.

5. Failure Analysis

- **Zero generalization to unseen poses.** Seen split avg max_reward = 0.80; unseen = 0.00. The LLM performs demo-conditional pattern matching — when the query's KP layout is close to a demo it copies enough waypoints to make contact, but held-out box poses break this entirely.
- **Scaling demos does not rescue unseen.** 5 → 10 → 20 in-context demos all give 0/5 unseen. Not starved for demonstrations; starved for in-context reasoning.
- **Scaling visual context does not help.** K = 5, 10, 20 anchored keypoints all give 0/5 unseen.
- **Action precision bounded by runtime.** M=10/20 produce well-formed JSON with 0% success. M=40 deterministically breaks the JSON parser — the runtime limit is hit before reasoning is the bottleneck.
- **Model scale at this tier doesn't matter.** llama3.2 (~3B) and gemma4:26b (~26B) are both below the GPT-4-class threshold required by KAT.
- **Open-loop rollout with single-frame observation.** All KPs are extracted at t=0 and M waypoints are linearly upsampled without visual feedback; trajectory drift accumulates uncorrected.

6. Discussion — What Works, What Doesn't

Component	Status
Data collection (seed-0, 25 episodes, downsample=10)	✓ Deterministic, manifest-documented
Subset splits (demos_5/10/20, test_5)	✓ Deterministic, hard-linked from base/
DINO ViT-S/16 anchored keypoints + serialization	✓ KP overlay confirms stable anchors
Action quantizer (per-dim uniform, bins=32)	✓ Round-trip succeeds on test episode
Ollama prompt + JSON schema	✓ parse_ok = 100% at M=10, 20
Simulator + replay + env_state0 propagation	✓ gt_replay = 100%
ICL reasoning at 3–26B local model scale	✗ Unseen success 0/5 every configuration

The negative result is informative: **the engineering chain is correct**, but the LLM tier currently runnable through local Ollama is not sufficient for KAT-style ICL on transfer cube. The fully instrumented pipeline allows a future user to (a) drop in a GPT-4-class model, (b) plug the prompt template into a different runtime, or (c) iterate on the action representation — all without touching the env / dataset / sweep code.

7. Reproducibility

Full sweep from a uv-activated venv (four commands):

```
source .venv/bin/activate
python3 scripts/download_dino_weights.py
python3 record_sim_episodes.py --task_name sim_transfer_cube_scripted \
```

```
--dataset_dir data/transfer_cube/base --num_episodes 25 --downsample_rate 10 --seed 0
python3 scripts/make_subsets.py --base_dir data/transfer_cube/base
python3 scripts/evaluate.py sweep --collection_seed 0 \
--models 'gemma4:26b,llama3.2:latest'
```

Or run **KAT_ICL.ipynb** top to bottom. All results CSVs and videos are saved under **artifacts/**.